

BABAPPA: a standalone simulation-trained method for BABAPPA-native branch-site support from user-supplied codon MSAs and trees

Krishnendu Sinha

Department of Zoology, Jhargram Raj College, Jhargram, West Bengal, India

Correspondence: dr.krishnendusinha@gmail.com

Abstract

Branch-site positive-selection analysis asks whether specific sites experienced episodic selection on specific phylogenetic branches. Standard likelihood tools such as codeml/PAML and HyPhy remain central references, but many practical analyses begin with a curated codon multiple sequence alignment (MSA) and a treefile and require a direct answer: which site on which branch is supported? BABAPPA (Branch-site Alignment-Bias-Aware Probabilistic Positive-selection Analyzer) is a research-alpha, simulation-trained method designed for that interface. The core command accepts only a user-supplied aligned codon MSA, a matching tree, and a foreground definition (`all`, `leaves`, or specific tip labels). It treats the supplied MSA as the authoritative homology statement, performs no mandatory realignment, and reports branch-site scores, calls, aligned codon coordinates, and branch-specific de-gapped codon coordinates.

BABAPPA is intentionally standalone. It does not call codeml or HyPhy to obtain its inference, and it does not borrow their null models. Instead, it reports BABAPPA-native evidence: a deployable neural branch-site scorer trained on explicit simulator branch-site truth, an applicability/out-of-domain gate, and an empirical feature-null calibration layer that returns p-like BABAPPA quantities and support classes. codeml and HyPhy are used in this manuscript only as optional publication comparators, not as dependencies or ground truth.

The current implementation, v0.7.0, is based on a conservative explicit branch-truth Apple Silicon/MPS validation at 100,000 simulated families. The retained 100K validation is a conditional pass because raw and intermediate files were intentionally pruned after completion, but retained summaries, audits, model artifacts, calibrations, controls, and stage markers support the completed run. Branch-site neural AUROC across low, moderate, high, and extreme saturation tiers was 0.998605, 0.996962, 0.994527, and 0.989955. Branch-level AUROC was 0.999797, 0.999421, 0.998777, and 0.997873. Gene-level AUROC was 0.999697, 0.999616, 0.999508, and 0.999430. These values describe simulation-supervised validation, not empirical false-discovery rates.

A close-taxa WRKY pilot demonstrates the present empirical behavior. `WRKY_candidate_01` was rejected as an out-of-domain stress test with mean p-distance 0.725799. `WRKY_candidate_02_close`, a seven-taxon close Brassicaceae `WRKY33/AT2G38470` panel, was in-domain and BABAPPA diagnostic-positive, with max gene support 0.177189 and 6,954 called branch-site rows. codeml and HyPhy comparators were negative, while a 100-replicate feature-level BABAPPA matched-null calibration showed unusual called-row burden but not unusual maximum gene support. The result is

therefore BABAPPA-only with mixed native-null support, not a cross-method biological discovery claim. The manuscript presents BABAPPA as a standalone complementary method whose outputs can be reported as BABAPPA-native calibrated branch-site support when QC, applicability, calibration, and biological context are documented.

1 Introduction

Branch-site positive-selection inference is central to molecular evolution because adaptation may affect only a subset of sites on a subset of branches. Classical codon-model tools estimate shifts in nonsynonymous-to-synonymous rate ratio, $\omega = d_N/d_S$, and include PAML/codeml and HyPhy workflows such as aBSREL and MEME [5, 8, 9, 11–13, 16, 19, 20]. These likelihood approaches are mature and interpretable, but they are not the only possible evidence system. They also remain sensitive to alignment quality, taxon sampling, saturation, recombination, hidden paralogy, synonymous-rate variation, multinucleotide substitutions, and model misspecification [1–3, 6, 10, 14, 15, 17, 18].

BABAPPA was developed for a deliberately simple user-facing problem. A user has an orthologous codon MSA and a treefile. The user wants a program that asks for those two files, asks which foreground branches to evaluate, and reports which site on which branch is supported as episodic selection. The software should not require the user to track internal simulation files, alignment ensembles, raw tensors, or external likelihood tools before obtaining a BABAPPA result. The user-supplied MSA is treated as the biological homology hypothesis. If a user wants alignment-sensitivity analysis, BABAPPA can provide optional workflows, but the default prediction path does not realign the input.

This manuscript describes BABAPPA as a standalone complementary method. “Standalone” means that BABAPPA can produce its own evidence statement without codeml or HyPhy. “Complementary” means that BABAPPA does not claim likelihood-model equivalence and should be compared with classical tools in publication benchmarking. BABAPPA’s evidence is simulation-trained and BABAPPA-native: it uses a deployable branch-site neural model, conservative feature policy, applicability/OOD checks, and empirical feature-null calibration. A manuscript may therefore report BABAPPA-native calibrated support, but should not call BABAPPA p-like values codeml/HyPhy likelihood-ratio p-values.

2 Method Overview

The direct BABAPPA workflow has five stages (Figure placeholder: direct workflow schematic).

1. **Input:** aligned codon MSA FASTA and Newick treefile with matching labels.
2. **Foreground selection:** all tips, leaves by default in the interactive interface, or user-specified tip labels.
3. **Feature extraction:** conservative branch-site features are generated from the user MSA and tree context without simulator truth.
4. **Scoring:** a deployable simulation-trained tier model scores each branch-site row.

5. **BABAPPA-native calibration:** an empirical branch-shuffle feature-null backend estimates p-like support values for the observed BABAPPA score burden.

The principal command is:

```
babappa predict-branch-sites -msa MSA -tree TREE -foreground all  
                             -null-replicates 1000
```

The interactive command **babappa** prompts for only the MSA path, treefile path, and foreground mode. It then uses the default native-null replicate count. The explicit command allows users to set **-null-replicates 0** for a quick uncalibrated check, **100** for pilot calibration, and **1000+** for manuscript-strength BABAPPA-native calibration.

3 Design Principles

3.1 The MSA supplied by the user is authoritative

In direct prediction mode, BABAPPA does not realign the input. It assumes the user has supplied the MSA they want interpreted. This design keeps the tool aligned with the practical use case: a researcher already has a curated ortholog-family MSA and tree, and wants branch-site predictions. BABAPPA still validates that the MSA is equal-length, codon-compatible, and label-compatible with the tree.

3.2 Simulator truth is never an empirical input

Simulation truth is used only during training and validation. Empirical inference rejects or audits truth-derived columns such as **branch_site_truth**, **selected_sites**, **truth**, **branch_truth**, **oracle**, **y_branch_site**, **y_site**, **gene_label**, **positive_label**, and **simulated_label**. This prevents the simulation-supervised training target from leaking into empirical scoring.

3.3 The feature policy is conservative

The deployable branch-site model uses the **conservative_branch_site** feature policy. This choice followed foreground-context ablation analyses showing shortcut risk from context-only features. The conservative policy is intended to preserve branch-site signal while reducing direct context leakage. It does not remove all model limitations, but it defines the safest current feature surface for research-alpha deployment.

3.4 Applicability and abstention are explicit outputs

BABAPPA evaluates whether a family is **in_domain**, **borderline**, or **out_of_domain**. The gate considers taxon count, codon count, pairwise divergence, saturation proxy, gap burden, ambiguity burden, site-map mappability, feature missingness, and available training-envelope

information. Out-of-domain cases are marked diagnostic-only and should be treated as stress tests rather than biological calls.

3.5 codeml and HyPhy are publication comparators, not dependencies

BABAPPA includes a separate `publication_benchmark/` pipeline for manuscript evaluation against codeml and HyPhy. That pipeline is not part of the core user-facing BABAPPA inference path. It exists to generate comparison tables for peer review, investigate failure modes, and contextualize BABAPPA-native evidence.

4 Software Implementation

BABAPPA is implemented as a Python command-line package. Table 1 summarizes the main components.

Table 1: BABAPPA software components.

Component	Role
Direct predictor	Scores a user-supplied codon MSA and tree without realignment; writes branch-site, branch, gene, QC, and native-null reports.
Feature extractor	Generates conservative branch-site features matching the deployable package schema.
Deployable scorer	Loads tier-specific neural checkpoints, calibration metadata, feature means/standard deviations, and threshold policies.
BABAPPA-native null	Shuffles branch/foreground feature columns to construct empirical feature-null replicates and computes p-like BABAPPA values.
Applicability/OOD gate	Classifies empirical input as in-domain, borderline, or out-of-domain.
Simulation framework	Generates explicit branch-site truth datasets for model development and validation.
Alignment sensitivity tools	Optional workflows using identity, MAFFT, BABAPPAlign, and MUSCLE; PRANK and T-Coffee are diagnostic-only.
Publication benchmark pipeline	Separate USER-RUN scripts for comparing BABAPPA-native results with codeml and HyPhy on curated manuscript panels.
Storage maintenance	<code>babappa audit-storage</code> inventories reproducible heavy outputs and generates safe cleanup scripts.

5 Training and Validation

BABAPPA was trained and validated using explicit simulator branch-site truth. The final retained validation state is a conservative 100K Apple Silicon/MPS run with:

- 100,000 simulated families total;
- four saturation tiers: low, moderate, high, extreme;
- 25,000 families per tier;
- explicit branch-site truth;
- `conservative_branch_site` feature policy;
- production aligners: identity, MAFFT [7], BABAPPAlign, and MUSCLE [4];
- streamed/capped branch-site datasets;
- MPS-compatible training and scoring infrastructure.

The validation decision is a conditional pass. The condition is due to storage management: raw and intermediate simulation/alignment/tensor directories were intentionally pruned after completion. Retained summaries, truth audits, model artifacts, calibrations, aggregation outputs, controls, and stage markers support the completed validation state. This condition should be reported whenever the 100K validation is cited.

Table 2: Conservative explicit branch-truth 100K validation metrics.

Saturation tier	Branch-site neural AUROC	Branch AUROC	Gene AUROC
Low	0.998605	0.999797	0.999697
Moderate	0.996962	0.999421	0.999616
High	0.994527	0.998777	0.999508
Extreme	0.989955	0.997873	0.999430

The smooth decline across saturation tiers supports the feasibility of the conservative branch-site pathway under the simulator. These results should not be interpreted as empirical false-discovery rates. Empirical interpretation requires direct input QC, applicability/OOD assessment, and BABAPPA-native calibration on the user’s family.

6 Deployable Model Package

The deployable package is:

`deployable_model_conservative_branch_site_100k_mps`

Package validation status is ok, with zero failures and zero warnings. The package includes `model_manifest.json`, `model_card.md`, `feature_schema.json`, `calibration_schema.json`, `training_envelope.json`, `tier_models/`, `tier_calibrations/`, `checksums.sha256`, `validation_summary.json`, `limitations.md`, and `README.md`. It excludes raw simulator truth, raw oracle labels, branch-site datasets, raw simulations, and alignments.

7 Direct Prediction Outputs

The direct command writes:

- `branch_site_predictions.tsv`: one row per scored branch-site combination;
- `branch_predictions.tsv`: branch-level support summary;
- `gene_summary.tsv`: gene-level BABAPPA summary;
- `prediction_report.md`: human-readable interpretation;
- `qc_report.md`: input and applicability summary;
- `babappa_native_null/`: native-null distributions and observed-versus-null summaries when `-null-replicates > 0`.

Branch-site prediction rows include both alignment coordinates and de-gapped coordinates. The key fields are:

- `msa_codon_site`: one-based codon column in the supplied MSA;
- `aligned_codon_site`: equivalent aligned codon coordinate;
- `branch_degapped_codon_site`: one-based codon coordinate in the scored branch after removing gapped codons;
- `branch_codon`: codon carried by the scored branch at the aligned site.

For manuscript reporting, the most important gene-level fields are `result_class`, `babappa_native_result_class`, `babappa_native_evidence_class`, `babappa_native_null_replicates`, `p_babappa_called_rows`, `p_babappa_max_gene_support`, `p_babappa_max_branch_support`, `applicability_status`, and `tier_model`.

8 BABAPPA-Native Null Calibration

BABAPPA’s standalone evidence layer is an empirical feature-null backend. For each replicate, branch- and foreground-specific feature columns are shuffled across the empirical feature table while preserving the overall feature surface and family size. The shuffled replicate is scored through the same deployable model and threshold policy as the observed family. BABAPPA then compares observed metrics with the null distribution:

- maximum gene support;
- maximum branch support;
- number of called positive branch-site rows;
- maximum site score;
- high-score site-score quantiles.

The p-like right-tail value is computed as

$$p_{\text{BABAPPA}} = \frac{1 + \#\{x_{\text{null}} \geq x_{\text{obs}}\}}{N_{\text{null}} + 1}.$$

BABAPPA classifies native-null evidence as `strong_babappa_native_support`, `babappa_native_support`, `not_significant_under_babappa_native_null`, or `underpowered_native_null`. The combined result class is stored as `babappa_native_result_class`. These values are BABAPPA-native and should be reported as such. They are not codeml/HyPhy likelihood-ratio p-values.

9 Publication Benchmarking Against codeml and HyPhy

For publication, BABAPPA includes a separate benchmark harness under `publication_benchmark/`. This harness runs BABAPPA-native calibrated prediction, prepares codeml and HyPhy reference folders, optionally runs those tools, parses outputs, and writes comparison tables. It is not used by BABAPPA’s normal inference command.

The benchmark is intended for curated manuscript panels with known or literature-supported positives, conserved negatives, alignment-sensitive families, saturated families, short/low-information families, and paralogy-risk families. codeml and HyPhy are interpreted as external comparators. They are not assumed to be ground truth, and BABAPPA is not tuned to mimic them.

10 Empirical WRKY Pilot

Two WRKY-family empirical pilots illustrate the current method behavior.

Table 3: WRKY empirical pilot outcomes.

Family	BABAPPA status	Interpretation
WRKY_candidate_01	OOD stress test; mean p-distance 0.725799; applicability out_of_domain; diagnostic_only=True; no positive call.	Useful failure-mode probe showing that highly divergent WRKY panels should be rejected or treated as diagnostic-only.

WRKY_candidate _02_close	Close-taxa WRKY33/AT2G38470 homologs; 7 Brassicaceae/close plant taxa; aligned mean p- distance approximately 0.101293; prefilter accept ; applicability in_domain ; moderate tier; MPS scoring; diagnostic- positive; max gene support 0.177189; 6,954 called branch- site rows; 10,451 score rows; diagnostic_only=False .	Demonstrates that the empir- ical bridge can produce gated BABAPPA-native scores on plausible biological inputs.
-----------------------------	--	---

For WRKY_candidate_02_close, codeml branch-site Model A versus null gave LRT 0.0 and $p = 1.0$, negative. HyPhy aBSREL gave foreground $p = 1.0$, negative. HyPhy MEME gave minimum $p = 0.0641705$, negative at 0.05. BABAPPA’s 100-replicate feature-level matched-null calibration gave `p_empirical_called_rows=0.009900990099009901` but `p_empirical_support=1.0` and `p_empirical_branch_support=1.0`. Thus the current WRKY result is BABAPPA-only with mixed BABAPPA-native support. It is useful method evidence, but not a cross-method biological discovery claim.

11 Limitations

BABAPPA is research-alpha and has explicit limitations.

- The deployable model is simulation-trained, so simulator-to-real transfer must be studied empirically.
- BABAPPA-native p-like values are not likelihood-ratio p-values and should not be described as codeml/HyPhy equivalents.
- A raw diagnostic-positive score without native-null calibration is not sufficient for manuscript interpretation.
- OOD families, high saturation, hidden paralogy, tree/MSA mismatch, or low mappability can invalidate biological interpretation.
- The 100K validation is a conditional pass because raw intermediates were pruned after completion.
- External disagreement should be interpreted as method contrast, not automatic truth or failure.

12 Reproducibility and Availability

The source version described here is v0.7.0. The release archive label is v0.7.0. The recorded archive is:

```
BABAPPA_v0.7.0_release_zenodo_[DATE].tar.xz  
SHA256: cc259617f19d9634fd6e11906903910498ab78d3797a10df1bb24b7db014dc30
```

The code repository will be deposited at [PLACEHOLDER_REPOSITORY_URL]. The Zenodo DOI placeholder is [PLACEHOLDER_ZENODO_DOI]. Large raw simulations, alignments, tensor shards, and streamed datasets are not intended for version control; retained reproducibility artifacts include model manifests, checksums, final reports, validation summaries, evidence packs, and publication benchmark scripts.

13 Conclusion

BABAPPA now matches the original software goal: from a user-supplied codon MSA and treefile, it reports branch-site support, branch-level summaries, gene-level summaries, de-gapped site coordinates, QC/applicability status, and BABAPPA-native calibrated evidence. It is a standalone complementary method, not a wrapper around codeml or HyPhy. codeml and HyPhy remain valuable external comparators for publication benchmarking, but BABAPPA's own inference is based on its simulation-trained deployable model and native empirical feature-null calibration.

Author contributions

Krishnendu Sinha conceived the project, designed the framework, implemented and validated the research-alpha workflow with AI-assisted software-development support, curated the empirical pilot state, and wrote the manuscript.

Competing interests

The author declares no competing interests.

Data and code availability

The code repository will be deposited at [PLACEHOLDER_REPOSITORY_URL]. The release archive label is v0.7.0 and the current source version described here is v0.7.0. The archive filename and SHA256 checksum are reported in the reproducibility section above and should be replaced by a DOI-resolved archive at [PLACEHOLDER_ZENODO_DOI] when the formal release is deposited.

References

- [1] Maria Anisimova, Rasmus Nielsen, and Ziheng Yang. Effect of recombination on the accuracy of the likelihood method for detecting positive selection at amino acid sites. *Genetics*, 164(3):1229–1236, 2003.
- [2] David A. Duchene, Lindell Bromham, and Sebastien Duchene. Excluding loci with substitution saturation improves phylogenomic inference. *Systematic Biology*, 71(3):676–689, 2022.
- [3] Katherine A. Dunn, Toby Kenney, Hong Gu, and Joseph P. Bielawski. Improved inference of site-specific positive selection under a generalized parametric codon model when there are multinucleotide mutations and multiple nonsynonymous rates. *BMC Evolutionary Biology*, 19(1):22, 2019.
- [4] Robert C. Edgar. MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5):1792–1797, 2004.
- [5] Nick Goldman and Ziheng Yang. A codon-based model of nucleotide substitution for protein-coding dna sequences. *Molecular Biology and Evolution*, 11(5):725–736, 1994.
- [6] Gregory Jordan and Nick Goldman. The effects of alignment error and alignment filtering on the sitewise detection of positive selection. *Molecular Biology and Evolution*, 29(4):1125–1139, 2012.
- [7] Kazutaka Katoh and Daron M. Standley. MAFFT multiple sequence alignment software version 7: improvements in performance and usability. *Molecular Biology and Evolution*, 30(4):772–780, 2013.
- [8] Sergei L. Kosakovsky Pond, Ben Murrell, Mathieu Fourment, Simon D. W. Frost, Wayne Delpont, and Konrad Scheffler. A random effects branch-site model for detecting episodic diversifying selection. *Molecular Biology and Evolution*, 28(11):3033–3043, 2011.
- [9] Sergei L. Kosakovsky Pond, Art F. Y. Poon, Ryan Velazquez, Steven Weaver, Nathan L. Hepler, Ben Murrell, Stephen D. Shank, Brittany R. Magalis, Dave Bouvier, Anton Nekrutenko, Sadie Wisotsky, Stephanie J. Spielman, Simon D. W. Frost, and Spencer V. Muse. HyPhy 2.5—a customizable platform for evolutionary hypothesis testing using phylogenies. *Molecular Biology and Evolution*, 37(1):295–299, 2020.
- [10] Anthony G. Lucaci, Sadie R. Wisotsky, Stephen D. Shank, Spencer V. Muse, and Sergei L. Kosakovsky Pond. Evolutionary shortcuts via multi-nucleotide substitutions and their impact on natural selection analyses. *Molecular Biology and Evolution*, 40(7):msad150, 2023.
- [11] Ben Murrell, Steven Weaver, Martin D. Smith, Joel O. Wertheim, Sasha Murrell, Amy Aylward, Kadri Eren, Thomas Pollner, Daniel P. Martin, David M. Smith, Konrad Scheffler, and Sergei L. Kosakovsky Pond. Gene-wide identification of episodic selection. *Molecular Biology and Evolution*, 32(5):1365–1371, 2015.

- [12] Ben Murrell, Joel O. Wertheim, Sasha Moola, Tim Weighill, Konrad Scheffler, and Sergei L. Kosakovsky Pond. Detecting individual sites subject to episodic diversifying selection. *PLoS Genetics*, 8(7):e1002764, 2012.
- [13] Spencer V. Muse and Brandon S. Gaut. A likelihood approach for comparing synonymous and nonsynonymous nucleotide substitution rates, with application to the chloroplast genome. *Molecular Biology and Evolution*, 11(5):715–724, 1994.
- [14] Benjamin D. Redelings. Erasing errors due to alignment ambiguity when estimating positive selection. *Molecular Biology and Evolution*, 31(8):1979–1993, 2014.
- [15] Konrad Scheffler, Darren P. Martin, and Cathal Seoighe. Robust inference of positive selection from recombining coding sequences. *Bioinformatics*, 22(20):2493–2499, 2006.
- [16] Martin D. Smith, Joel O. Wertheim, Steven Weaver, Ben Murrell, Konrad Scheffler, and Sergei L. Kosakovsky Pond. Less is more: an adaptive branch-site random effects model for efficient detection of episodic diversifying selection. *Molecular Biology and Evolution*, 32(5):1342–1353, 2015.
- [17] Aarti Venkat, Matthew W. Hahn, and Joseph W. Thornton. Multinucleotide mutations cause false inferences of lineage-specific positive selection. *Nature Ecology & Evolution*, 2:1280–1288, 2018.
- [18] Sadie R. Wisotsky, Sergei L. Kosakovsky Pond, Stephen D. Shank, and Spencer V. Muse. Synonymous site-to-site substitution rate variation dramatically inflates false positive rates of selection analyses: ignore at your own peril. *Molecular Biology and Evolution*, 37(8):2430–2439, 2020.
- [19] Ziheng Yang. PAML 4: phylogenetic analysis by maximum likelihood. *Molecular Biology and Evolution*, 24(8):1586–1591, 2007.
- [20] Jianzhi Zhang, Rasmus Nielsen, and Ziheng Yang. Evaluation of an improved branch-site likelihood method for detecting positive selection at the molecular level. *Molecular Biology and Evolution*, 22(12):2472–2479, 2005.